

Indirect Tax Classification & Reconciliation Engine - Recommendation Memo

Why this project

Built to close a specific, named gap on the JPMorgan Chase - Business Analyst, Indirect Tax Operations application: the role sits inside Payment Operations / Global Supplier Services, applying the correct VAT/GST/Sales & Use Tax/withholding treatment to every vendor invoice the firm pays, globally. That team's own stated failure mode - *"a wrong or stale tax code is the single most common root cause of a tax ops exception queue"* - is the mechanic this project builds, not a generic tax calculator.

Methodology

1. **Rules sourcing** (data/raw/tax_rules.csv) - 20 rules across 11 jurisdictions, each row carrying a source, an as_of_date, and a confidence flag.
2. **Synthetic invoice generation** (src/generate_synthetic_invoices.py) - 60 lines across all 11 jurisdictions, 3 per scenario, with 1-in-3 lines deliberately given a wrong or blank vendor-master tax code.
3. **Classification** (src/classification_engine.py) - independently re-derives the correct tax code per line and compares it against the (possibly corrupted) vendor-master code.

India's GST rules reflect the actual 4-slab structure (0% / 5% / 18% / 40%) that took effect 22 Sept 2025, replacing the older slab-with-cess system - using the pre-reform slabs would have been a real, checkable error. No real vendor or transaction data exists anywhere for this project - every invoice line is synthetic. Three classification outcomes are possible: OK (clean match, confirmed rate), OK - VERIFY RATE (match, but the underlying rate wasn't independently re-confirmed this session), EXCEPTION (code mismatch, blank code, or no matching rule at all).

Results

Jurisdiction	Lines	Exceptions	Exception rate
India	30	10	33.3%
EU (6 jurisdictions)	18	6	33.3%
US (4 states)	12	4	33.3%
Total	60	20	33.3%

Every jurisdiction lands on exactly 33.3% - the deterministic 1-in-3 injection rate - which is the validation gate, not a coincidence: it confirms the engine correctly detects every injected error and produces zero false positives on clean lines.

What broke

The first run reported a 66.7% exception rate - implausibly high, and the first sign something was wrong rather than a genuinely bad tax posture. Root cause: generate_synthetic_invoices.py used the rules table's tax_type column value ("Sales & Use Tax", "VAT") in the field the classification engine matches on category ("State base rate", "Standard") - so every US and EU line failed to find any matching rule and was misclassified as a missing-rule exception, regardless of whether its vendor-master code was actually correct. Fixed by aligning the generator's labels to the rules table's real category column. Re-run produced the expected, deterministic 33.3% - the kind of exact-match validation gate that catches this class of bug immediately rather than letting a plausible-looking wrong number ship.

Limitations

See LIMITATIONS.md. In short: 9 of 20 rates are well-established facts not independently re-confirmed against a live primary source this session (flagged, not hidden); no real transaction data exists to validate against; US rates are state-level base only (no local/county add-ons); India TDS/withholding rates should be re-checked against the current Finance Act before any real filing use, given the ongoing Income-tax Act 2025 renumbering to Section 393.

What this demonstrates for the role

The core operational skill the JD tests for - apply a rule set correctly to a transaction and flag what doesn't fit - reconciled against a deterministic, checkable outcome, the same discipline as the RBI ECL engine reproducing a bank's own reported provisioning number. This is not a claim of tax domain expertise; it is evidence of the reconciliation habit the role runs on.